Lecture 12: Modern DB Architectures & NoSQL

Amit Kumar Dhar

September 3, 2025

Database Application Classes

OLTP vs. OLAP

Databases typically serve two main application classes:

OLTP (Online Transaction Proc.)

- **Focus:** Day-to-day operations.
- **Workload:** Fast, atomic transactions (read/write/update).
- **Data:** Current, normalized.
- **Example:** E-commerce purchase, ATM withdrawal, flight booking.

OLAP (Online Analytical Proc.)

- **Focus:** Business intelligence and decision support.
- **Workload:** Complex, aggregate queries on large datasets.
- **Data:** Historical, denormalized.
- **Example:** Quarterly sales report, market trend analysis.

The Rise of NoSQL

Motivation for NoSQL

The demands of modern web-scale applications revealed limitations in traditional RDBMS:

- **Massive Scale:** Hard to scale horizontally (across many cheap servers). Vertical scaling (bigger servers) is expensive.
- **Schema Flexibility:** Requires a predefined schema.
- **High Availability & Fault Tolerance:** Often have a single point of failure.

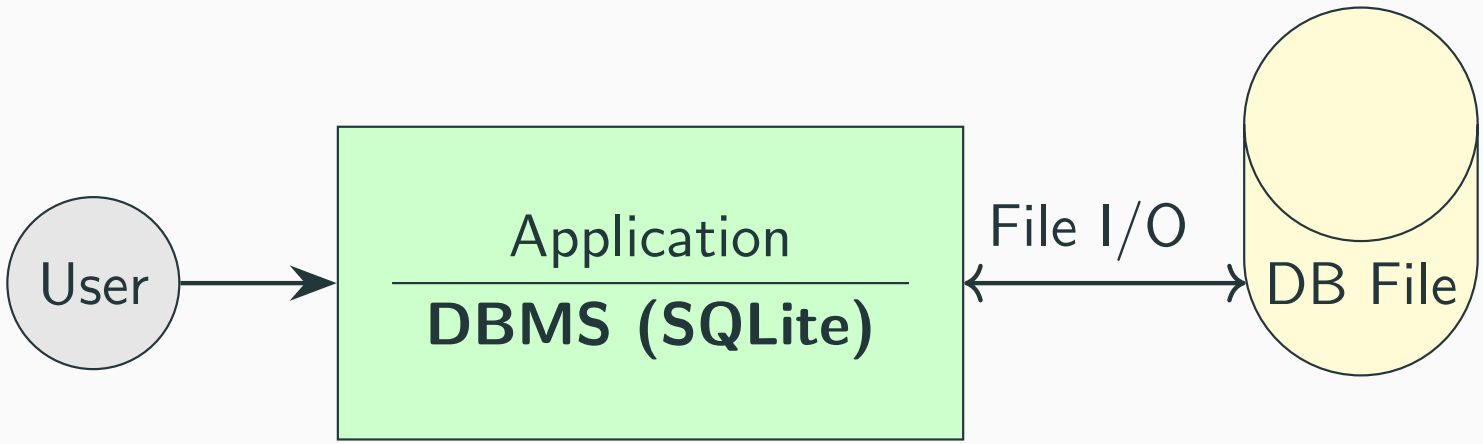
Real-world drivers:

- **Facebook:** Storing petabytes of user profiles and social graph data.
- **Amazon:** Managing shopping carts and product catalogs with extreme uptime requirements.
- **Google:** Indexing the entire web and managing massive datasets.

RDBMS Architecture Review

Architecture 1: Serverless (Embedded)

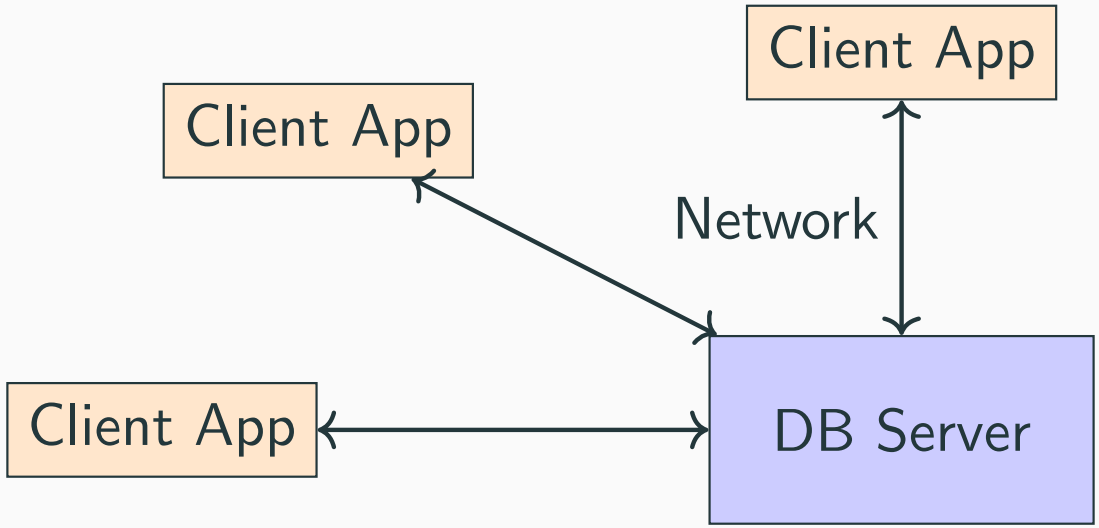
The DBMS is a library linked directly into the application, operating on a local file.



- **Consistency:** Easy to maintain. All constraints are handled within a single process.
- **Application:** Limited. Ideal for single-user applications (e.g., mobile apps, desktop software) but not for concurrent multi-user access.

Architecture 2: Client-Server

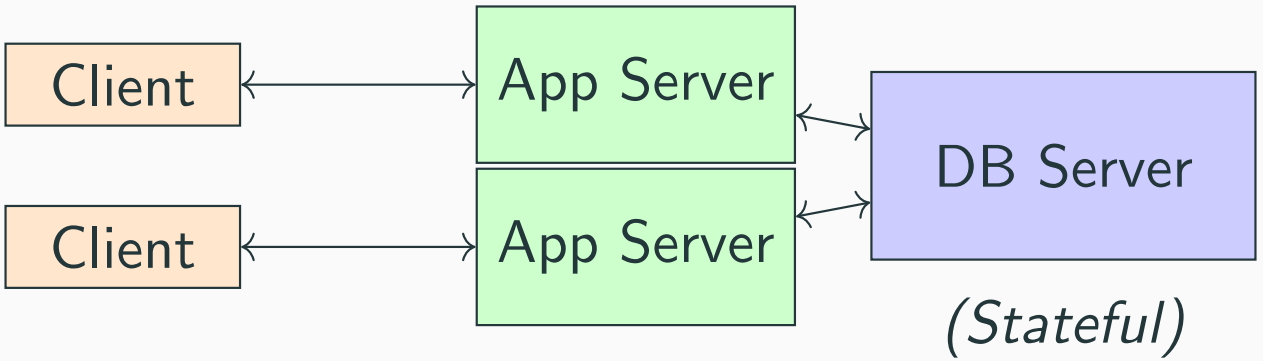
A central database server manages data and accepts network connections from multiple clients.



- **Consistency:** Harder to manage. The server must handle concurrent transactions from many clients, requiring complex locking and isolation mechanisms.
- **Bottleneck:** The DB server can become a performance bottleneck.

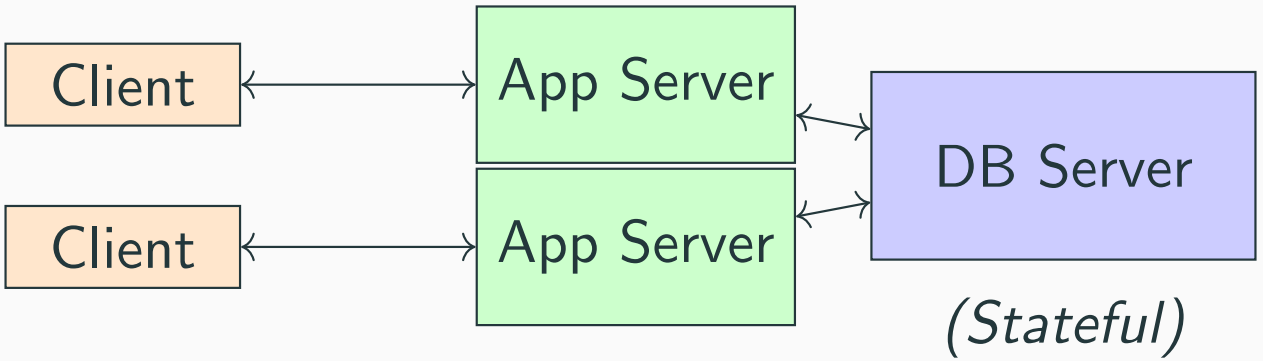
Architecture 3: 3-Tier Web Apps

Decouples presentation (client), logic (app server), and data (DB server).



Architecture 3: 3-Tier Web Apps

Decouples presentation (client), logic (app server), and data (DB server).

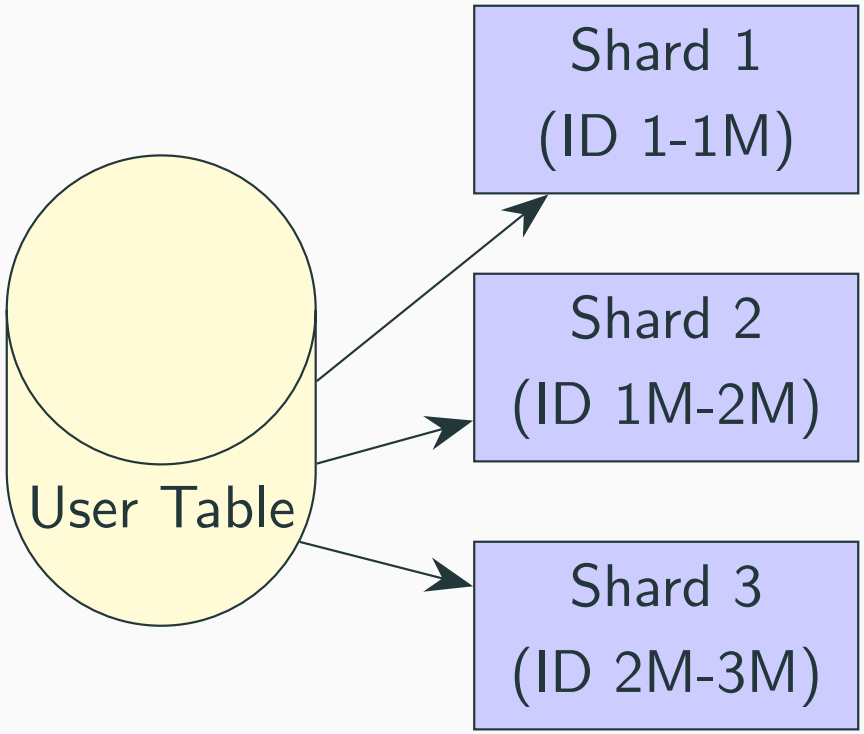


- Application servers are **stateless** and can be easily replicated.
- The DB server is **stateful** (it holds the data)
- Replication of DB is complex because all replicas must be kept consistent.

Scaling Databases

Scaling through Partitioning (Sharding)

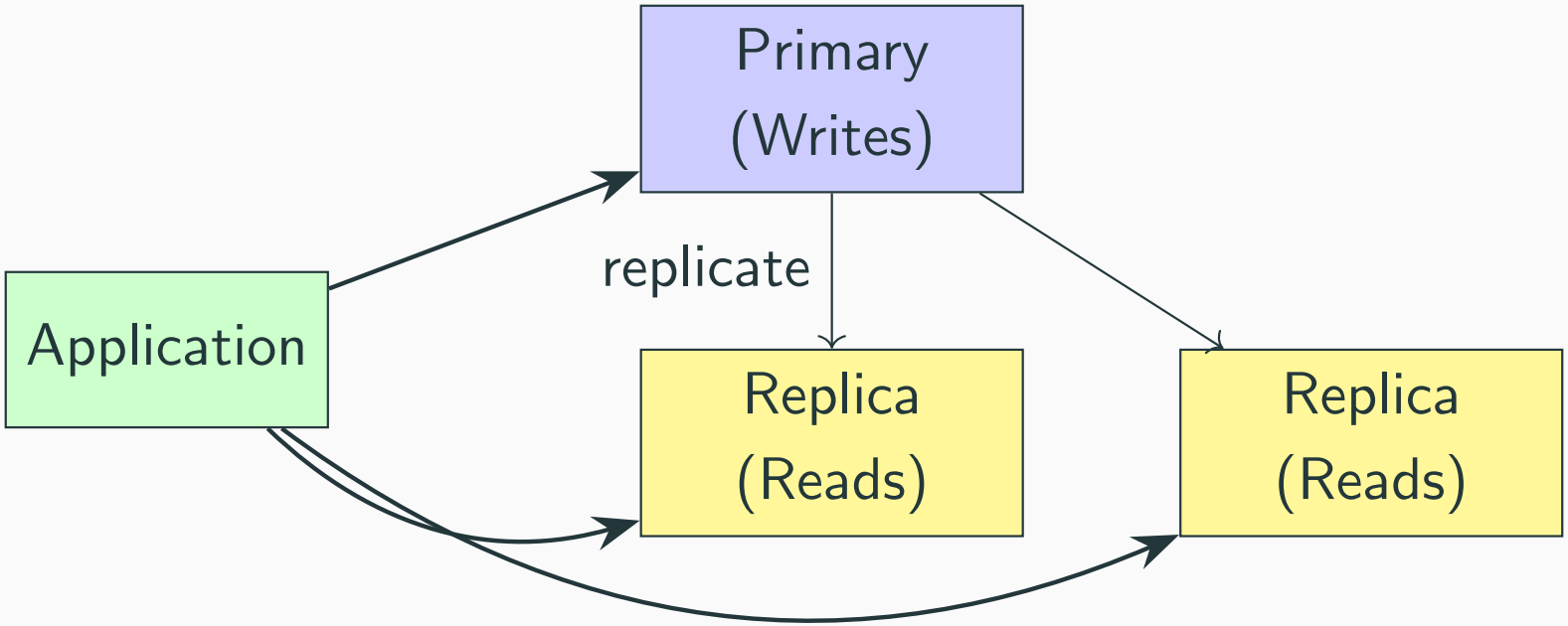
Data is split horizontally across multiple servers based on a "shard key".



- **Advantages:** Distributes write load, scales storage capacity.
- **Disadvantages:** Joins across partitions are very expensive. Transactions across partitions require complex coordination.

Scaling through Replication

Copies of the database are created to handle more read requests and provide high availability.



- **Advantages:** Improves read performance, provides fault tolerance (if primary fails, a replica can be promoted).
- **Disadvantages:** Writes are still a bottleneck. "Replication lag" can lead to stale data on replicas (eventual consistency).

RDBMS vs. NoSQL

Key Differences

Feature	Relational (RDBMS)	NoSQL
Partitioning	Difficult, manual	Automatic (sharding)
Replication	Master-Slave common	Built-in, often multi-master
Consistency	Strong (ACID)	Tunable (Eventual - BASE)
Joins	Core feature, efficient	Often unsupported or slow
Schema	Rigid, predefined	Dynamic, flexible

NoSQL Data Models

Key-Value Stores

- **Data Model:** A simple dictionary or hash map. The value is an opaque blob to the database.
- **Operations:** `get(key)`, `put(key, value)`, `delete(key)`.
- **Distribution:** The key is hashed to determine which partition stores the data.
- **Examples:** Redis, Amazon DynamoDB, Riak.

Relational to Key-Value Example: A student row ‘(101, ’Alice’, ’CS’)‘ becomes a key-value pair:

- **Key:** ‘student:101‘
- **Value:** ‘”{”sname”: ”Alice”, ”major”: ”CS”}‘ (as a JSON string)

Document Stores

Motivation: An evolution of Key-Value stores where the database understands the structure of the value (the document).

- **Data Model:** A collection of structured documents (e.g., JSON, BSON).
- **Operations:** `get`, `put`, plus rich querying capabilities on fields inside the document.
- **Distribution:** Typically partitioned by a document ID (key).
- **Advantages:** Data locality (related data is in one document), flexible schema, maps well to application objects.
- **Examples:** MongoDB, Couchbase.

Example: Modeling Facebook Friends

Relational Model: Requires a join table.

```sql
-- Users(user_id, name)
-- Friendships(user1_id, user2_id)
SELECT u2.name FROM Users u1
JOIN Friendships f ON u1.user_id = f.user1_id
JOIN Users u2 ON f.user2_id = u2.user_id
WHERE u1.name = 'Alice';
```

Key-Value Model: Requires multiple application-side lookups.

```
// 1. Get Alice's user object
get("user:101") -> "{'name':'Alice', 'friends':[102, 105]}"

// 2. In application, loop and get each friend
get("user:102") -> "{'name':'Bob'}"
get("user:105") -> "{'name':'Charlie'}"
```

Example: Modeling Facebook Friends

Document Model: Data can be embedded for fast, single-query retrieval.

```
// Get Alice's document, friends are embedded
db.users.find({name: "Alice"}) ->
{
  _id: 101, name: "Alice",
  friends: [
    {id: 102, name: "Bob"},
    {id: 105, name: "Charlie"}
  ]
}
```

Thank You!

Questions?